PROJETO INTEGRADOR / FIAP

MISSÃO AURORA SIGER

FASE 02

MGPEB

Módulo de Gerenciamento de Pouso e Estabilização de Base

Relatório técnico do protótipo de gerenciamento de cargas destinadas à implantação robótica de infraestrutura em Marte.

7 MÓDULOS **1 ÁREA DE POUSO** **1 RODADA**

Wilton Junior

Ciência da Computação • FIAP

14 de outubro de 2026

CÓDIGO-FONTE E TESTES / GITHUB

github.com/wiltonjr4/projeto-aurora-siger

Acesso à pasta fase_02_mgpeb pelo link ou QR code.

02 / CONTEXTO DA MISSÃO

Escopo, dados de entrada e premissas do Módulo de Gerenciamento de Pouso e Estabilização de Base.

01 Problema e objetivo

A Missão Aurora Siger propõe, em cenário hipotético, a chegada de sete módulos de carga destinados à **implantação robótica de infraestrutura em Marte**, antes da presença de tripulação. O MGPEB é um protótipo em Python que organiza a ordem de análise, verifica condições mínimas de segurança e registra cada decisão.

A atmosfera marciana é rarefeita e há tempestades de poeira, o que motiva a atenção a sensores, ambiente e segurança operacional. Essas características ambientais são referências reais; **os dados e limites usados na simulação são inteiramente fictícios** [1].

02 Inventário dos módulos

ID	Carga / finalidade	Prior.	Comb.	Massa kg	Chegada
AS-001	Reconhecimento / comunicação	1	78%	4.500	09:30
AS-002	Energia	2	65%	6.000	10:15
AS-003	Infraestrutura / robótica	3	72%	5.500	10:50
AS-004	Produção / recursos	4	68%	4.200	11:30
AS-005	Habitação	5	82%	9.000	12:10
AS-006	Suprimentos / suporte médico	6	74%	5.000	11:05
AS-007	Laboratório científico	7	88%	3.800	10:40

Criticidade: alta nos módulos AS-001 a AS-006 e média no AS-007. Todos partem com *sensores_integros = True* e *status = em_orbita*; somente o cenário de demonstração injeta falha de sensor no AS-002.

03 Premissas de operação

- **Uma única área de pouso:** as decisões são sequenciais, sem pousos simultâneos.
- **Prioridade de 1 a 7:** o menor número indica atendimento preferencial, não dispensa verificações.
- **Horários padronizados:** comparação em HH:MM, com todos os eventos no mesmo dia.
- **Rodada única:** módulos em espera ou alerta permanecem registrados para análise posterior, ainda não automatizada.

03 / ARQUITETURA DO SOFTWARE

Separação entre cadastro, algoritmos, motor de decisão e apresentação dos resultados.

04 Fluxo de processamento



A função **executar_simulacao()** prepara um novo cadastro para cada chamada. **processar_fila()** retira um elemento por vez, avalia as condições, atualiza o estado e adiciona um registro ao histórico. **mostrar_resultados()** apenas apresenta os dados: a exibição não modifica a simulação.

05 Algoritmos e complexidade

Estrutura / algoritmo	Implementação	Custo típico
FIFO	**pop(0)** na lista de espera operacional	O(n) por retirada
Busca linear	ID, menor combustível e criticidade	O(n)
Bubble Sort	Comparação de prioridades adjacentes	$O(n^2)$, pior caso
Pilha LIFO	append() nos registros; pop() na consulta	O(1), operações no topo
Estados	pousado, em_espera e em_alerta	O(1) por atualização

06 Por que essas escolhas?

A frota reduzida (sete módulos) permite privilegiar **clareza didática e verificabilidade**. O Bubble Sort evidencia comparações e trocas e a fila com *pop(0)* demonstra diretamente a disciplina FIFO. Para uma missão com muitos eventos, seria desejável considerar *deque*, filas de prioridade eficientes, persistência e validações adicionais.

A fila operacional usa referências aos dicionários do cadastro; mudanças de status são visíveis no registro central. Cópias dos dados iniciais preservam uma “fotografia” para apresentação, e cada execução começa com um cadastro novo.

04 / PORTAS LÓGICAS E AUTORIZAÇÃO

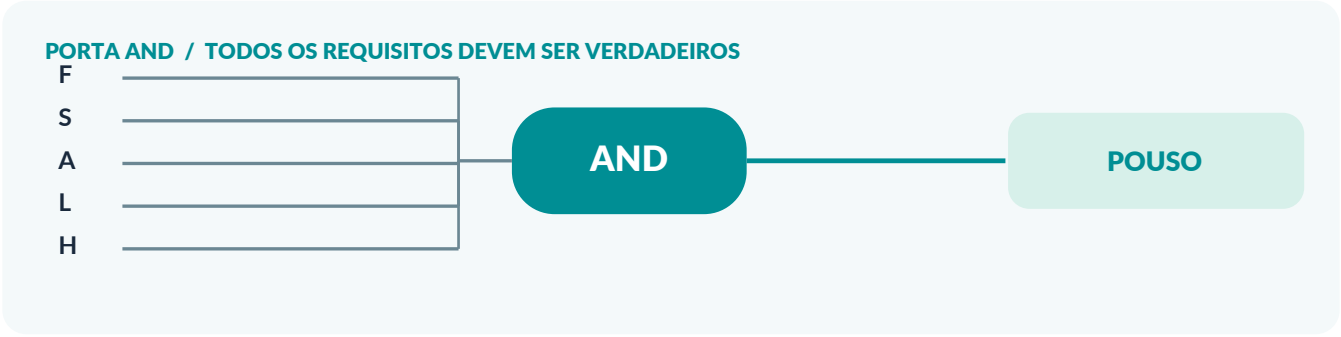
Formalização das verificações implementadas em avaliar_pouso(), no arquivo mgpeb.py.

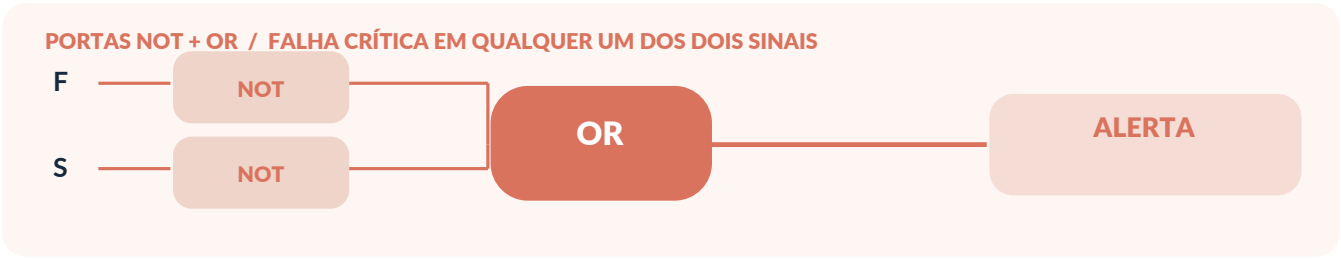
07 Variáveis booleanas

F: combustível ≥ 40% (limite didático) **S:** sensores íntegros **A:** atmosfera segura
L: área de pouso liberada **H:** horário previsto já atingido.

$$POUSO = F \wedge S \wedge A \wedge L \wedge H \qquad ALERTA = \neg F \vee \neg S$$

08 Circuitos conceituais





09 Decisão prioritária e exemplos

A função verifica **primeiro o alerta**. Se não houver falha crítica, concede autorização somente quando as cinco variáveis forem verdadeiras. Caso contrário, registra **em_espera**. Portanto, o mesmo módulo não recebe dois destinos na rodada.

F	S	A	L	H	Decisão
1	1	1	1	1	Pousado
1	0	1	1	1	Em alerta
0	1	1	1	1	Em alerta
1	1	0	1	1	Em espera
1	1	1	0	1	Em espera
1	1	1	1	0	Em espera

05 / MODELAGEM MATEMÁTICA

Modelo linear complementar ao código: ilustração da altitude em uma etapa idealizada da descida.

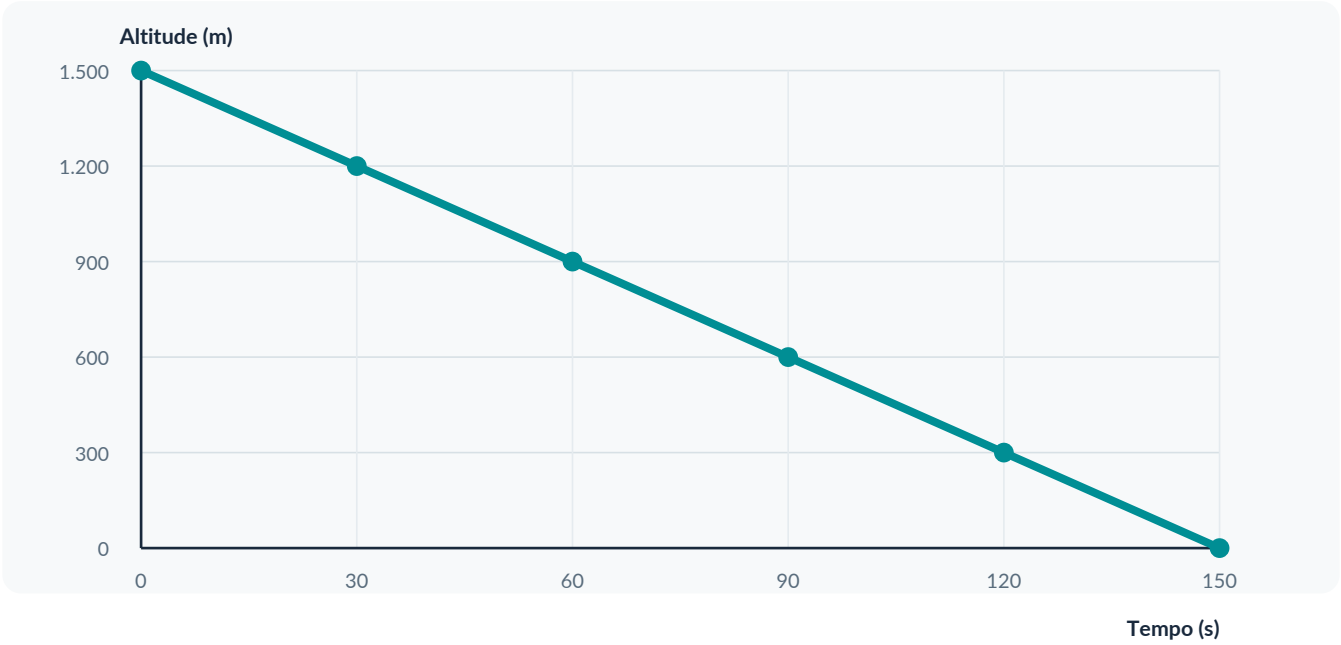
10 Função linear de altitude

Para visualizar a relação entre tempo e altitude, considera-se um trecho hipotético de descida vertical com velocidade constante. Não é uma previsão da trajetória real em Marte, pois ignora gravidade variável com o movimento, arrasto, propulsão, massa de propelente e controles de estabilidade.

$$h(t) = 1.500 - 10t \qquad 0 \le t \le 150 \text{ s}$$

h em metros; t em segundos; variação constante de −10 m/s.

11 Gráfico da função



Leitura: a cada 30 s, a altitude reduz 300 m. No modelo, o trecho começa a 1.500 m e termina em altitude zero após 150 s. O coeficiente angular é −10 m/s; o intercepto vertical é 1.500 m.

12 Valores calculados e vínculo com a missão

Tempo (s)	0	30	60	90	120	150
h(t), em m	1.500	1.200	900	600	300	0

No MGPEB, a modelagem numérica **efetivamente implementada** é a comparação **combustivel_percentual ≥ 40** e o teste do horário **chegada_prevista ≤ hora_atual**. A função de altitude e seu gráfico são uma extensão conceitual do relatório; **não foram integrados ao Python** e não constituem um controle de navegação.

06 / HISTÓRIA E ARQUITETURA COMPUTACIONAL

Da computação embarcada da era Apollo aos princípios de confiabilidade de sistemas espaciais.

13 Linha histórica e evolução técnica

1960–1970

Apollo Guidance Computer

Computação embarcada para navegação, orientação e controle; hardware e memória limitados [2].

Décadas seguintes

Computadores em sondas

Expansão de sensores, microprocessadores e software de missão; necessidade de robustez ambiental.

Era dos rovers

Perseverance / RAD750

Redundância e eletrônica resistente à radiação; visão computacional especializada na descida [3].

MGPEB / FIAP

Protótipo educacional

Python em computador convencional demonstra políticas e registros, sem requisitos de tempo real.

14 Componentes de uma arquitetura de voo

Elemento	Papel em um sistema espacial real
Processador	Executa ciclo de controle; precisa manter previsibilidade e tratar falhas.
Memória	Armazena software e estados, com medidas para integridade dos dados.
Sensores / atuadores	Fornecem sinais físicos e executam comandos de navegação e controle.
Comunicação	Troca telemetria e comandos, sem depender de resposta imediata da Terra.
Redundância	Permite caminhos alternativos diante de falhas de componentes.

15 Relação crítica com o software desenvolvido

O MGPEB representa somente uma **camada lógica de planejamento e classificação**. Ele não lê telemetria física, não opera com tempo real determinístico, não possui comunicação espacial, controle de propulsão, redundância de hardware ou certificação de software crítico. Em um projeto real, a autorização deveria ser verificada independentemente por subsistemas embarcados de segurança.

07 / ESG E GOVERNANÇA MARCIANA

Princípios ambientais, sociais e de governança aplicados à exploração robótica responsável.

16 Sustentabilidade como requisito de projeto

A expansão da presença humana e robótica além da Terra exige planejamento que preserve a integridade da pesquisa e reduza riscos evitáveis. A **proteção planetária** é uma referência concreta: controlar a contaminação introduzida por espaçonaves terrestres ajuda a proteger locais de interesse científico [4].

E

AMBIENTAL

- Evitar dispersão desnecessária de resíduos e materiais durante operações.
- Planejar pousos em regiões avaliadas quanto ao valor científico e ao impacto ambiental.
- Considerar energia, poeira e possíveis riscos de contaminação.

S

SOCIAL

- Projetar decisões conservadoras para proteger futuras equipes e a missão científica.
- Documentar alarmes de forma compreensível para supervisão humana.
- Garantir transparência na comunicação de falhas e limites dos sistemas.

G

GOVERNANÇA

- Preservar histórico auditável de cada análise e seu motivo.
- Impedir que a prioridade suplante critérios mínimos de segurança.
- Estabelecer revisão, responsabilidades e rastreabilidade para incidentes.

17 O que existe no código e o que é proposto

Situação	Prática ESG / governança
Já implementado	Registro de ID, destino e motivo; bloqueio por sensor e combustível; classificação por estado.
Melhoria proposta	Controle de resíduos, medição energética, proteção planetária, revisão humana e rastreabilidade persistente.
Indicadores sugeridos	Percentual de decisões registradas; alertas revisados; consumo de energia por operação; resíduos produzidos.

Na rodada demonstrada, **7 de 7 decisões geram eventos em memória**. Isso comprova rastreabilidade durante a execução, mas não constitui registro permanente nem auditoria externa. Indicadores ambientais e sociais acima são propostas, **não métricas medidas pelo programa**.

08 / RESULTADOS E VALIDAÇÃO

Cenário padrão efetivamente reproduzido no mgpeb.py e verificado por test_mgpeb.py.

18 Condições configuradas às 11:00

Hora: 11:00 | **Atmosfera:** segura | **Área:** disponível | **Combustível mínimo:** 40%
Falha de demonstração: sensores do AS-002 configurados como não íntegros. O cenário foi preparado para exercitar os três resultados possíveis.

03 POUSADOS

03 EM ESPERA

01 EM ALERTA

19 Resultado por módulo

Módulo	Situação	Justificativa
AS-001	Pousado	Todas as condições atendidas
AS-002	Em alerta	Sensores sem integridade
AS-003	Pousado	Todas as condições atendidas
AS-004	Em espera	Horário previsto: 11:30
AS-005	Em espera	Horário previsto: 12:10
AS-006	Em espera	Horário previsto: 11:05
AS-007	Pousado	Todas as condições atendidas

20 Testes automatizados realizados

O arquivo **test_mgpeb.py** utiliza *assert* para testar o comportamento sem dependências externas: cadastro, busca por ID existente/inexistente, menor combustível, criticidade, fila vazia, lista invertida, preservação da entrada, limiar exato de 40%, baixo combustível, falha dos sensores, área ocupada, atmosfera desfavorável, chegada futura, integridade dos destinos, LIFO e independência de rodadas.

TESTES: TODOS PASSARAM — python test_mgpeb.py

Interpretação: a fila encerra a rodada vazia porque todos os sete pedidos foram avaliados uma vez. Módulos adiados ou em alerta continuam em listas próprias; **isso não significa que todos tenham pousado**. O histórico contém sete eventos e pode ser consultado em ordem LIFO.

09 / APÊNDICE, LIMITAÇÕES E FONTES

Orientações de reprodução, síntese das estruturas lineares e referências para avaliação docente.

A Apêndice técnico: estruturas de dados

Elemento	Papel no protótipo
Cadastro	Lista de sete dicionários; mantém atributos e estado atual.
Fila de pouso	Lista ordenada por prioridade; retirada FIFO por pop(0).
Listas de destino	Três listas disjuntas no cenário: pousados, em espera, em alerta.
Pilha de eventos	Lista append()/pop(): consulta o registro mais recente primeiro.
Cópias	copy() preserva ordem da lista; dicionários de execução compartilham referência.

B Como reproduzir e consultar o código

```
python mgpeb.py

python test_mgpeb.py
```

Bibliotecas externas: nenhuma.

REPOSITÓRIO PÚBLICO / FASE 02

github.com/wiltonjr4/projeto-aurora-siger

Pasta: fase_02_mgpeb • arquivos mgpeb.py, test_mgpeb.py e README.md

C Limitações e escopo dos resultados

Não são simulados dinâmica orbital, pouso físico, estabilização real, comunicação com atraso, ocupação temporal da área, sensores reais, persistência em disco nem reavaliação automática dos módulos adiados. O rótulo **pousado** equivale à autorização considerada concluída na simulação. O limite de combustível, os horários, as massas e os estados são valores acadêmicos, não parâmetros certificados.

D Referências técnicas

[1] NASA. **Mars: Facts**. Características ambientais e atmosfera de Marte. science.nasa.gov/mars/facts/

[2] NASA. **Computers in Spaceflight: Apollo Guidance Computer (Hardware)**. www.nasa.gov/wp-content/uploads/static/history/computers/CH2-5.html

[3] NASA/JPL. **Mars 2020 Perseverance Landing Press Kit**. Computação e redundância. mars.nasa.gov/system/downloadable_items/45385_mars_2020_landing_press_kit.pdf

[4] NASA/JPL. **Planetary Protection: Mission Implementation**. planetaryprotection.jpl.nasa.gov/mission-implementation